

Skill week-06

WEEK-6:

Task 1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Task 2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

nano task1_escape.c

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 200
```

```
/*
```

```
 * Process escape sequences.
```

```
 *
```

```
 * Supported:
```

```
 * \ -> backslash
```

```
 * \ -> escaped space
```

```
 * \! -> escaped !
```

```
 * \$ -> escaped $
```

```
 * \" -> escaped double quote
```

```
 * \' -> escaped single quote
```

```
 */
```

```
void process_escape(const char *input, char *output) {
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    while (input[i] != '\0') {
```

```
        if (input[i] == '\\') {
```

```
            /* There is a character after the backslash */
```

```
            if (input[i + 1] != '\0') {
```

```
                i++;
```

```
            /*
```

```
             * Remove the escape character and
```

```
             * preserve the character after it.
```

```
            */
```

```
            output[j++] = input[i];
```

```
            i++;
```

```
        } else {
```

```
            /* Preserve a trailing backslash */
```

```
            output[j++] = '\\';
```

```
            i++;
```

```
        }
```

```
    } else {
```

```
        output[j++] = input[i];
```

```
        i++;
```

```
    }
```

```
}
```

```

    output[j] = '\0';
}

void test_case(const char *input) {
    char output[MAX];

    process_escape(input, output);

    printf("Input : %s\n", input);
    printf("Output: %s\n", output);
    printf("-----\n");
}

int main() {

    printf("WEEK 6 - TASK 1\n");
    printf("Escape Sequence Processing\n\n");

    /* Test 1: Escaped space */
    test_case("Hello\\ World");

    /* Test 2: Escaped special character */
    test_case("Hello\\!World");

    /* Test 3: Escaped dollar sign */
    test_case("Price\\$100");

    /* Test 4: Escaped double quote */
    test_case("He\\\"said");

    /* Test 5: Escaped single quote */
    test_case("It\\'s");

    /* Test 6: Multiple escape sequences */
    test_case("Hello\\ World\\! Test\\$100");

    /* Test 7: Complex input */
    test_case("C:\\\\Users\\\\Bharath\\\\Documents");

    /* Test 8: Normal input */
    test_case("Operating Systems");

    printf("\nValidation completed successfully.\n");

    return 0;
}

```

```

bharath@DESKTOP-6D34C2R:~/skill/week6$ nano task1_escape.c
bharath@DESKTOP-6D34C2R:~/skill/week6$ gcc task1_escape.c -o
bharath@DESKTOP-6D34C2R:~/skill/week6$ ./task1
WEEK 6 – TASK 1
Escape Sequence Processing

Input : Hello\ World
Output: Hello World
-----
Input : Hello\!World
Output: Hello!World
-----
Input : Price\$100
Output: Price$100
-----
Input : He\"said
Output: He"said
-----
Input : It\'s
Output: It's
-----
Input : Hello\ World\! Test\$100
Output: Hello World! Test$100
-----
Input : C:\\Users\\Bharath\\Documents
Output: C:\Users\Bharath\Documents
-----
Input : Operating Systems
Output: Operating Systems
-----

Validation completed successfully.
bharath@DESKTOP-6D34C2R:~/skill/week6$ echo Hello\ World
Hello World
bharath@DESKTOP-6D34C2R:~/skill/week6$ echo Price\$100
Price$100

```

```

nano task2_fork_exec.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

```

```

int main() {

    pid_t pid;
    int status;

```

```

printf("Parent process started.\n");
printf("Parent PID: %d\n\n", getpid());

/* Create child process */
pid = fork();

if (pid < 0) {
    perror("fork failed");
    return 1;
}

/* Child process */
if (pid == 0) {

    printf("Child process created.\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n\n", getppid());

    printf("Child is executing: ls -l\n\n");

    /*
     * Replace child process with ls program.
     * Arguments:
     * ls    -> program name
     * ls    -> argv[0]
     * -l    -> argument
     */
    execlp("ls", "ls", "-l", NULL);

    /*
     * execlp() returns only if execution fails.
     */
    perror("exec failed");

    exit(1);
}

/* Parent process */
else {

    printf("Parent waiting for child...\n\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {

        printf("\nChild finished.\n");
    }
}

```

```

        printf("Child exit status: %d\n",
               WEXITSTATUS(status));

    } else {
        printf("Child did not terminate normally.\n");
    }
}

return 0;
}

```

```

bharath@DESKTOP-6D34C2R:~/skill/week6$ nano task2_fork_exec.c
bharath@DESKTOP-6D34C2R:~/skill/week6$ gcc task2_fork_exec.c -o task2
bharath@DESKTOP-6D34C2R:~/skill/week6$ ./task2
Parent process started.
Parent PID: 1105

Parent waiting for child...

Child process created.
Child PID: 1106
Parent PID: 1105

Child is executing: ls -l

total 40
-rwxr-xr-x 1 bharath bharath 16128 Sep 20 05:44 task1
-rw-r--r-- 1 bharath bharath  1973 Sep 20 05:44 task1_escape.c
-rwxr-xr-x 1 bharath bharath 16352 Sep 20 05:48 task2
-rw-r--r-- 1 bharath bharath  1335 Sep 20 05:48 task2_fork_exec.c

Child finished.
Child exit status: 0
bharath@DESKTOP-6D34C2R:~/skill/week6$ |

```